Valid Bit	**Tag**	**PP number**
1	**11**	**12**
1	**7**	**4**
1	**3**	**6**
0	**4**	**9**

Valid Bit	**PP number**
1	**5**
0	**Δίσκος**
0	**Δίσκος**
1	**6**
1	**9**
1	**11**
0	**Δίσκος**
1	**4**
0	**Δίσκος**
0	**Δίσκος**
1	**3**
1	**12**

Σημείωση: Όταν χρειάζεται να μεταφερθεί πληροφορία από τον Δίσκο στην κύρια μνήμη, θα επιστρέφεται ο μικρότερος ελεύθερος αριθμός για PP number.

A) Για σελίδες των 4KB απαιτείται page offset = 12 bits (log2(4×1024)=12bits). Επομένως, το tag θα αποτελείται από τα υπόλοιπα bits του κάθε virtual address.

9452 = 0010 0100 1110 1100 , tag = 0010 = 2

είναι TLB miss και page table miss(page fault)

οπότε πάμε στον δίσκο για την διεύθυνση.

Έστω ότι επιστρέφεται το 0 ως PP number

Αντικαθιστάτε το #3 στο TLB αφού είναι invalid

LRU σειρά: #0, #1, #2, #3

Valid Bit	Tag	PP number
1	11	12
1	7	4
1	3	6
1	2	0

Valid Bit	PP number
1	5
0	Δίσκος
1	0
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

30964 = 0111 1000 1111 0100, tag = 0111 = 7

είναι TLB hit στο #1

επομένως δεν αλλάζει κάτι στους πίνακες.

LRU σειρά: #0, #2, #3, #1.

Valid Bit	Tag	PP number
1	11	12
1	7	4
1	3	6
1	2	0

Valid Bit	PP number
1	5
0	Δίσκος
1	0
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

19136 = 0100 1010 1101 1011 , tag = 0100 = 4

είναι TLB miss αλλά page table hit

Επιστρέφεται το 9 ως PP number

Αντικαθιστάτε το #0 στο TLB

LRU σειρά: #2, #3, #1, #0

Valid Bit	Tag	PP number
1	4	9
1	7	4
1	3	6
1	2	0

Valid Bit	PP number
1	5
0	Δίσκος
1	0
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

46502= 1011 0101 1010 0110, tag = 1011 = 11

είναι TLB miss αλλά page table hit.

Επιστρέφεται το 12 ως PP number

Αντικαθιστάτε το #2 στο TLB

LRU σειρά: #3, #1, #0, #2

Valid Bit	Tag	PP number
1	4	9
1	7	4
1	11	12
1	2	0

Valid Bit	PP number
1	5
0	Δίσκος
1	0
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

38110= 1001 0100 1101 1110, tag = 1001 = 9

είναι TLB miss και page table miss(page fault)

οπότε πάμε στον δίσκο για την διεύθυνση.

Έστω ότι επιστρέφεται το 1 ως PP number

Αντικαθιστάτε το #3 στο TLB

LRU σειρά: #1, #0, #2, #3

Valid Bit	Tag	PP number
1	4	9
1	7	4
1	11	12
1	9	1

Valid Bit	PP number
1	5
0	Δίσκος
1	0
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
1	1
1	3
1	12

16653= 0100 0001 0000 1101, tag = 0100 = 4

είναι TLB hit στο #0

επομένως δεν αλλάζει κάτι στους πίνακες

LRU σειρά: #1, #2, #3, #0

Valid Bit	Tag	PP number
1	4	9
1	7	4
1	11	12
1	2	0

Valid Bit	PP number
1	5
0	Δίσκος
1	0
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
1	1
1	3
1	12

48480 = 1011 1101 0110 0000, tag = 1011 = 11 είναι TLB hit στο #2 επομένως δεν αλλάζει κάτι στους πίνακες, LRU σειρά: #1, #3, #0, #2

ΤΕΛΙΚΟΣ ΠΊΝΑΚΑΣ

Valid Bit	**Tag**	**PP number**
1	**4**	**9**
1	**7**	**4**
1	**11**	**12**
1	**9**	**1**

Valid Bit	**PP number**
1	**5**
0	**Δίσκος**
1	**0**
1	**6**
1	**9**
1	**11**
0	**Δίσκος**
1	**4**
0	**Δίσκος**
1	**1**
1	**3**
1	**12**

Valid Bit	**Tag**	**PP number**
1	**11**	**12**
1	**7**	**4**
1	**3**	**6**
0	**4**	**9**

Valid Bit	**PP number**
1	**5**
0	**Δίσκος**
0	**Δίσκος**
1	**6**
1	**9**
1	**11**
0	**Δίσκος**
1	**4**
0	**Δίσκος**
0	**Δίσκος**
1	**3**
1	**12**

B) Για σελίδες των 16KB απαιτείται page offset = 14bits (log2(16×1024)=14bits). Επομένως, το tag θα αποτελείται από τα υπόλοιπα bits του κάθε virtual address.

9452 = 0010 0100 1110 1100 , tag = 00 = 0

είναι TLB miss αλλά page table hit.

Επιστρέφεται το 5 ως PP number.

Αντικαθιστάτε το #3 στο TLB αφού είναι invalid

LRU σειρά: #0, #1, #2, #3

30964 = 0111 1000 1111 0100, tag = 01 = 1

είναι TLB miss και page table miss(page fault)

οπότε πάμε στον δίσκο για την διεύθυνση.

Έστω ότι επιστρέφεται το 0 ως PP number

Αντικαθιστάτε το #0 στο TLB

LRU σειρά: #1, #2, #3, #0

Valid Bit	Tag	PP number
1	11	12
1	7	4
1	3	6
1	0	5

Valid Bit	PP number
1	5
0	Δίσκος
0	Δίσκος
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

Valid Bit	Tag	PP number
1	1	0
1	7	4
1	3	6
1	0	5

Valid Bit	PP number
1	5
1	0
0	Δίσκος
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

19136 = 0100 1010 1101 1011 , tag = 01 = 1

είναι TLB hit στο #0

επομένως δεν αλλάζει κάτι στους πίνακες.

LRU σειρά: #1, #2, #3, #0

Valid Bit	Tag	PP number
1	1	0
1	7	4
1	3	6
1	0	5

Valid Bit	PP number
1	5
1	0
0	Δίσκος
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

46502= 1011 0101 1010 0110, tag = 10 = 2

είναι TLB miss και page table miss(page fault)

οπότε πάμε στον δίσκο για την διεύθυνση.

Έστω ότι επιστρέφεται το 1 ως PP number

Αντικαθιστάτε το #1 στο TLB

LRU σειρά: #2, #3, #0, #1

Valid Bit	Tag	PP number
1	1	0
1	2	1
1	3	6
1	0	5

Valid Bit	PP number
1	5
1	0
1	1
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

38110= 1001 0100 1101 1110, tag = 10 = 2

είναι TLB hit στο #1

επομένως δεν αλλάζει κάτι στους πίνακες.

LRU σειρά: #2, #3, #0, #1

Valid Bit	Tag	PP number
1	1	0
1	2	1
1	3	6
1	0	5

Valid Bit	PP number
1	5
1	0
1	1
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

16653= 0100 0001 0000 1101, tag = 01 = 1

είναι TLB hit στο #0

επομένως δεν αλλάζει κάτι στους πίνακες

LRU σειρά: #2, #3, #1, #0

Valid Bit	Tag	PP number
1	1	0
1	2	1
1	3	6
1	0	5

Valid Bit	PP number
1	5
1	0
1	1
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

48480 = 1011 1101 0110 0000, tag = 10 = 2 είναι TLB hit στο #1 επομένως δεν αλλάζει κάτι στους πίνακες,

LRU σειρά: #2, #3, #0, #1

ΤΕΛΙΚΟΣ ΠΊΝΑΚΑΣ

Valid Bit	**Tag**	**PP number**
1	**1**	**0**
1	**2**	**1**
1	**3**	**6**
1	**0**	**5**

Valid Bit	**PP number**
1	**5**
1	**0**
1	**1**
1	**6**
1	**9**
1	**11**
0	**Δίσκος**
1	**4**
0	**Δίσκος**
1	**1**
1	**3**
1	**12**

Πλεονεκτήματα Μεγαλύτερου Μεγέθους Σελίδας:

1. **Μικρότερος Πίνακας Σελίδων**:
   - Λιγότερες σελίδες για την ίδια περιοχή διευθύνσεων.
   - Μειώνεται η συνολική μνήμη που απαιτείται για τον πίνακα σελίδων.
2. **Λιγότερα TLB Misses**:
   - Κάθε TLB entry καλύπτει μεγαλύτερο χώρο μνήμης (16KB αντί για 4KB).
   - Μειώνεται η ανάγκη για συχνές ενημερώσεις του TLB.
3. **Απλούστερη Διαχείριση Δίσκου**:
   - Η φόρτωση μιας σελίδας από το δίσκο καλύπτει μεγαλύτερη περιοχή, μειώνοντας τα I/O operations.

Μειονεκτήματα Μεγαλύτερου Μεγέθους Σελίδας:

1. **Αυξημένη Εσωτερική Κατακερματισμός (Internal Fragmentation)**:
   - Αν μια διαδικασία χρειάζεται μικρότερη μνήμη από το μέγεθος της σελίδας, το υπόλοιπο μένει ανεκμετάλλευτο.
2. **Μεγαλύτερη Καθυστέρηση Ενημέρωσης Μνήμης**:
   - Η φόρτωση μιας μεγαλύτερης σελίδας από το δίσκο μπορεί να πάρει περισσότερο χρόνο.
3. **Δυσκολότερη Ευρεσιμότητα Ελεύθερων Χώρων**:
   - Οι μεγαλύτερες σελίδες μπορεί να δυσκολεύουν την εύρεση συνεχόμενων ελεύθερων φυσικών περιοχών.

Index	**Valid Bit**	**Tag**	**PP num**
0	**1**	**11**	**12**
1	**1**	**7**	**4**
2	**1**	**3**	**6**
3	**0**	**4**	**9**

Valid Bit	**PP number**
1	**5**
0	**Δίσκος**
0	**Δίσκος**
1	**6**
1	**9**
1	**11**
0	**Δίσκος**
1	**4**
0	**Δίσκος**
0	**Δίσκος**
1	**3**
1	**12**

C) Για σελίδες των 4KB απαιτείται page offset = 12bits (log2(4×1024)=12bits). Επίσης τα index bits πρέπει να είναι 2 (log2(4 είσοδοι στο TLB) = 2). Επομένως το tag θα είναι τα δύο upper bits.

Tag (2bits)	Index (2bits)	Offset (12bits)

9452 = 0010 0100 1110 1100 , tag = 0, index = 2

είναι TLB miss αλλά page table hit.

Επιστρέφεται το 5 ως PP number

στο #2, γιατί index = 2.

30964 = 0111 1000 1111 0100, tag = 1, index = 3

είναι TLB miss και page table miss(page fault)

οπότε πάμε στον δίσκο για την διεύθυνση.

Έστω ότι επιστρέφεται το 0 ως PP number

στο #3, γιατί index = 3.

Index	Valid Bit	Tag	PP num
0	1	11	12
1	1	7	4
2	1	0	5
3	0	4	9

Valid Bit	PP number
1	5
0	Δίσκος
0	Δίσκος
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

Index	Valid Bit	Tag	PP num
0	1	11	12
1	1	7	4
2	1	0	5
3	1	1	0

Valid Bit	PP number
1	5
1	0
0	Δίσκος
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

19136 = 0100 1010 1101 1011 , tag = 1, index = 0

είναι TLB miss αλλά page table hit.

Επιστρέφεται το 0 ως PP number .

στο #0, γιατί index = 0.

Index	Valid Bit	Tag	PP num
0	1	1	0
1	1	7	4
2	1	0	5
3	1	1	0

Valid Bit	PP number
1	5
1	0
0	Δίσκος
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

46502= 1011 0101 1010 0110, tag = 2, index = 3

είναι TLB miss και page table miss(page fault)

οπότε πάμε στον δίσκο για την διεύθυνση.

Έστω ότι επιστρέφεται το 1 ως PP number

στο #3, γιατί index = 3.

Index	Valid Bit	Tag	PP num
0	1	1	0
1	1	7	4
2	1	0	5
3	1	2	1

Valid Bit	PP number
1	5
1	0
1	1
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

38110= 1001 0100 1101 1110, tag = 2, index = 1

είναι TLB miss αλλά page table hit.

Επιστρέφεται το 1 ως PP number ..

στο #1, γιατί index = 1.

16653= 0100 0001 0000 1101, tag = 1, index = 0

είναι TLB hit στο #0

επομένως δεν αλλάζει κάτι στους πίνακες

Index	Valid Bit	Tag	PP num
0	1	1	0
1	1	2	1
2	1	0	5
3	1	2	1

Valid Bit	PP number
1	5
1	0
0	1
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

Index	Valid Bit	Tag	PP num
0	1	1	0
1	1	2	1
2	1	0	5
3	1	2	1

Valid Bit	PP number
1	5
1	0
1	1
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

48480 = 1011 1101 0110 0000, tag = 2, index = 3 είναι TLB hit στο #3 επομένως δεν αλλάζει κάτι στους πίνακες,

ΤΕΛΙΚΟΣ ΠΙΝΑΚΑΣ

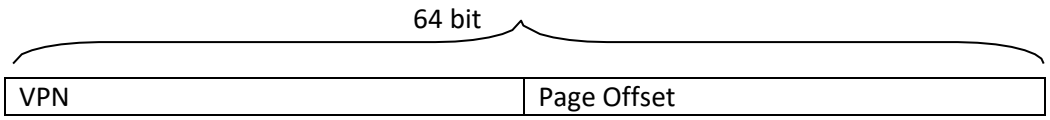
Index	**Valid Bit**	**Tag**	**PP num**
0	**1**	**1**	**0**
1	**1**	**2**	**1**
2	**1**	**0**	**5**
3	**0**	**2**	**1**

Valid Bit	**PP number**
1	**5**
0	**0**
0	**1**
1	**6**
1	**9**
1	**11**
0	**Δίσκος**
1	**4**
0	**Δίσκος**
0	**Δίσκος**
1	**3**
1	**12**

Σημασία Υψηλής Απόδοσης TLB:

1. **Μείωση Χρόνου Πρόσβασης στη Μνήμη**:
   - Το TLB λειτουργεί ως cache για τον πίνακα σελίδων (page table), επιτρέποντας ταχύτερη μετάφραση από εικονικές σε φυσικές διευθύνσεις.
   - Χωρίς το TLB, κάθε προσπέλαση στη μνήμη θα απαιτούσε αναζήτηση στον πίνακα σελίδων, κάτι που προσθέτει σημαντική καθυστέρηση (π.χ., αν ο πίνακας βρίσκεται στην κύρια μνήμη).
2. **Μείωση Σφαλμάτων (Misses)**:
   - Ένας TLB υψηλής απόδοσης έχει χαμηλό **miss rate**, που σημαίνει ότι οι περισσότερες προσπελάσεις βρίσκουν τις απαιτούμενες πληροφορίες στον TLB.
   - Μειώνεται η ανάγκη πρόσβασης στον πίνακα σελίδων.

D)

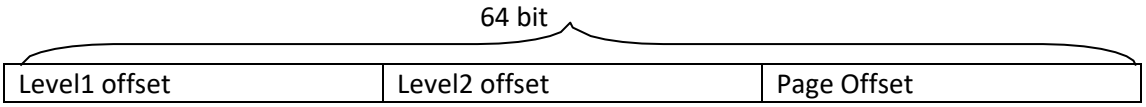


Για σελίδες των 16KB απαιτείται page offset = 14bits (log2(16×1024)=14bits). Επομένως, το VPN θα αποτελείται από τα υπόλοιπα bits του virtual address, VPN = 64 -14 = 50 bits. Άρα το μέγεθος τους page table είναι:

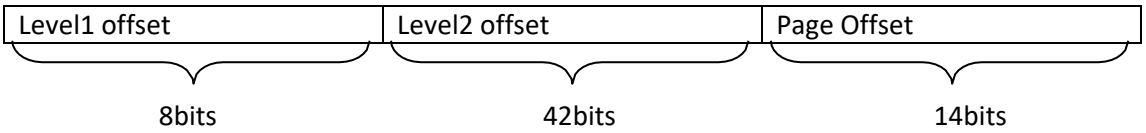
2^{50} * 8bytes (μέγεθος εισόδου) = 2^{53} (και αυτό μόνο για μία εφαρμογή)

Εφόσον κάθε εφαρμογή έχει ξεχωριστό χώρο διευθύνσεων, το συνολικό μέγεθος του πίνακα σελίδων για 5 εφαρμογές είναι ⟶ **5 * 2^{53-}**

E)



Το μέγεθος των σελίδων παραμένει 16KB επομένως το Page offset απαιτεί 14bits. Το πρώτο επίπεδο έχει 256 entries οπότε το level1 offset είναι 8bits (log2(256) = 8). Άρα το level2 offset πρέπει να είναι
64 – 14 – 8 = 42bits



Προκύπτει πως το μέγεθος του πρώτο επιπέδου είναι 2^8 * 8bytes(μέγεθος εισόδου) = 2^{11}
και το μέγεθος του δεύτερου επιπέδου είναι 2^{42} * 8bytes(μέγεθος εισόδου) = 2^{45} bytes
Επομένως το συνολικό μέγεθος είναι $2^{11} + 2^{45}$ για μια εφαρμογή δηλαδή για 5 εφαρμογές είναι **5 * ($2^{11} + 2^{45}$).**

****Παρατήρηση** : το μέγεθος είναι πολύ μικρότερο από το 5 * 2^{53} που βρέθηκε από το προηγούμενο ερώτημα, που το page table ήταν ενός επιπέδου. Για αυτό προτιμάτε η χρήση πολλαπλών επιπέδων.

F) Χρησιμοποιώντας την τεχνική virtually indexed and physically tagged θα δημιουργηθεί μια cache για την οποία ισχύει μέγεθος cache = μέγεθος σελίδας * cache associativity. Αυτό ισχύει, διότι για μια virtually indexed and physically tagged **direct-mapped** cache το index της προκύπτει εξ ολοκλήρου από το page offset. Επομένως, το συνολικό μέγεθος της cache περιορίζεται από το μέγεθος της σελίδας, που σημαίνει ότι μπορούμε να κατασκευάσουμε μια cache 16KB είναι (μέγεθος cache = 16KB * 1).

Για να αυξήσουμε το μέγεθος της cache αρκεί να μεγαλώσουμε το associativity της, για παράδειγμα:

- 2 way associativity, το μέγεθος της cache διπλασιάζεται ⟶ 32KB.
- 4 way associativity, το μέγεθος της cache τετραπλασιάζεται ⟶ 64KB.

ΑΣΚΗΣΗ 2

A)

Αρχικά στον πίνακα A[127] κάθε double καταλαμβάνει 8 bytes άρα ο κάθε πίνακας A ενός node καταλαμβάνει 127 * 8 = 1016 bytes. Επιπλέον, σε αρχιτεκτονική των 64-bit ένας pointer είναι 8bytes. Άρα το συνολικό μέγεθος του struct element είναι 1016 + 8 = 1024 = 1KB.

Το κάθε page έχει μέγεθος 4KB, αυτό σημαίνει πως σε ένα page θα αποθηκευτεί η πληροφορία 4 nodes(δηλαδή 4 στοιχείων). Υπάρχουν 256 στοιχεία (N = 256), ο αριθμός των σελίδων είναι 256/4 = 64, δηλαδή ανά 4 nodes θα προκύπτει 1 TLB miss γιατί προσπελαύνεται καινούργια σελίδα **(64 TLB misses)**.

Τα blocks της L1-D-cache είναι 32bytes το καθένα, αυτό σημαίνει πως στο ίδιο block χωράνε 4 συνεχόμενα στοιχεία του πίνακα A. Ο πίνακας A έχει 127 στοιχεία τα οποία θα συμπυκνωθούν σε $\lceil 127/4 \rceil = 32$ blocks. Το τελευταίο block (32) θα αποτελείται από το στοιχειό 125,126,127 και από το node->next(8bytes). Επομένως, δεν θα προκαλούνται misses από την εντολή node = node->next. Εφόσον ο κάθε κόμβος απαιτεί 32 blocks, θα δημιουργεί 32 cache misses. Υπάρχουν 256 στοιχεία, άρα προκύπτουν 256 * 32 = **8192 cache misses**.

B)

Το σύνολο των προσπελάσεων στην TLB είναι :

256 * 127(για τον πίνακα A) + 256(για τον επόμενο node) = 32768

Άρα το **miss rate της TLB** είναι 64/32768 = 0,0019… = **0,19%**

Το σύνολο των προσπελάσεων στην L1-D-cache είναι :

256 * 127(για τον πίνακα A) + 256(για τον επόμενο node) = 32768

Άρα το **miss rate της L1-D-cache** είναι 8192/32768 = 0,25 = **25%**

C)

Υπάρχουν πολλαπλές περιπτώσεις που μπορούν να επηρεάσουν τον ρυθμό του προγράμματος. Αυτές οι περιπτώσεις πρέπει να αναλυθούν ξεχωριστά.

- **TLB και cache hit:** Αυτό το ενδεχόμενο επαναλαμβάνεται 32768 - 64 - 8192 = 24512. Οι ποινές σε αυτή την περίπτωση είναι 1 κύκλος μηχανής για την TLB και την cache. **Επομένως, ο αριθμός των κύκλων είναι 24512**.
- **TLB Hit και cache miss:** Αυτό το ενδεχόμενο επαναλαμβάνεται 8192. Οι ποινές σε αυτή την περίπτωση είναι 50 από την cache. **Επομένως, ο αριθμός των κύκλων είναι 8192 * 50 = 409600.**
- **TLB Miss σελίδα στην κυρία μνήμη (cache miss):** Αυτό το ενδεχόμενο επαναλαμβάνεται 32 φορές, γιατί τις μισές φορές θα υπάρχει στην κύρια μνήμη αυτό που ψάχνεται αφού σε κάθε page fault ‘’το λειτουργικό σύστημα αναλαμβάνει να φέρει από τον σκληρό δίσκο όχι μόνο την σελίδα που δημιούργησε το σφάλμα, αλλά και την αμέσως επόμενη σελίδα’’. Οι ποινές σε αυτή την περίπτωση είναι 50 κύκλοι μηχανής από την TLB και μετά 50 από την cache. **Επομένως, ο αριθμός των κύκλων είναι 32 * 100 = 3200.**
- **TLB miss και page fault:** Αυτό το ενδεχόμενο επαναλαμβάνεται τις υπόλοιπες 32 φορές για τον λόγο που εξηγήθηκες πριν. Οι ποινές σε αυτή την περίπτωση είναι 10000 κύκλοι μηχανής από την TLB. **Επομένως, ο αριθμός των κύκλων είναι 32 * 10000 = 320000.**

Άρα το AMAT είναι = συνολικός αριθμός κύκλων μηχανής / προσπελάσεις στη μνήμη

AMAT = (24512 + 409600 + 3200 + 320000) / 32768

AMAT = 23,1.